

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

Contents



1. Workload Picked
2. Batch of requests through various Engines
3. LLM Request Timelines
4. NSYS Measurements
5. Throughput vs Load for vLLM
6. GPU Memory Breakdown
7. Future Steps

Workload Picked



- Targeted a very specific workload
- Large system prompt ($\approx$ 1500 tokens) containing useful context and shared across all the queries
- Varying query input rate (from 5 req/sec to 700 req/sec)
- Small query sizes (10 to 20 tokens each) and small output requirements (capped at 30 to 50 tokens)
- Ideally, prefix sharing should help a lot
- **Next Step:** Family of requests, each family with a particular large system prompt and a few queries

Batch of requests through various Engines

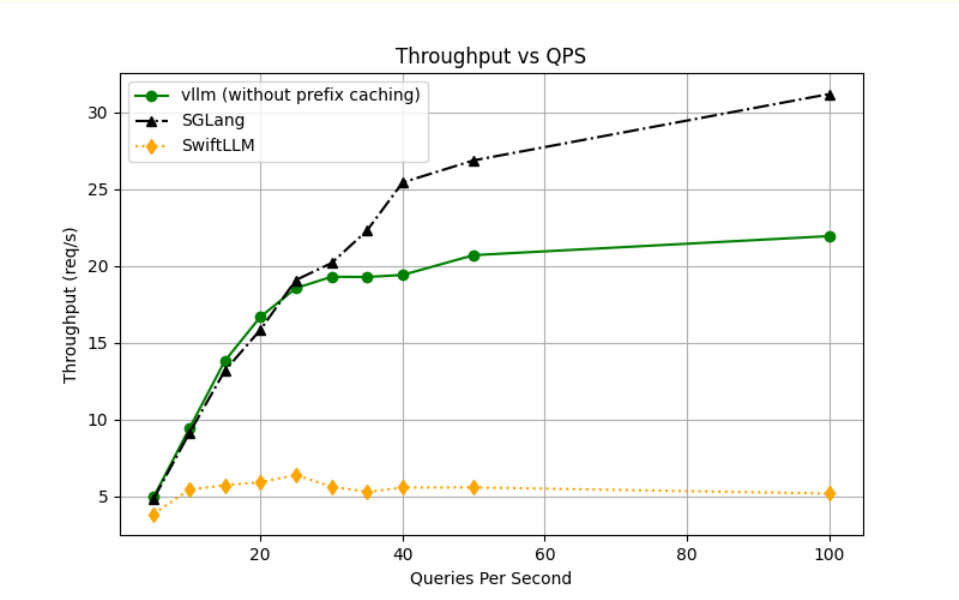


Technique Used	Time (s)
Transformers (batch size = 1)	30.21
Transformers (batch size = 2)	19.51
Transformers (batch size = 3)	18.64
vLLM (no system prompt pre-filling)	0.45
SGLang (fork)	1.03
SGLang (run_batch)	0.40
NEO	0.55

Table: Measuring the time taken to process a batch of 24 requests using different techniques

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

Batch of requests through various Engines



- SwiftLLM is a small copy (2000 lines of code) of vLLM, promises to perform equivalent to vLLM.
- SGLANG performs quite better because of prefix sharing through radix tree.

Figure: Throughput comparison across varying workloads

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

Batch of requests through various Engines

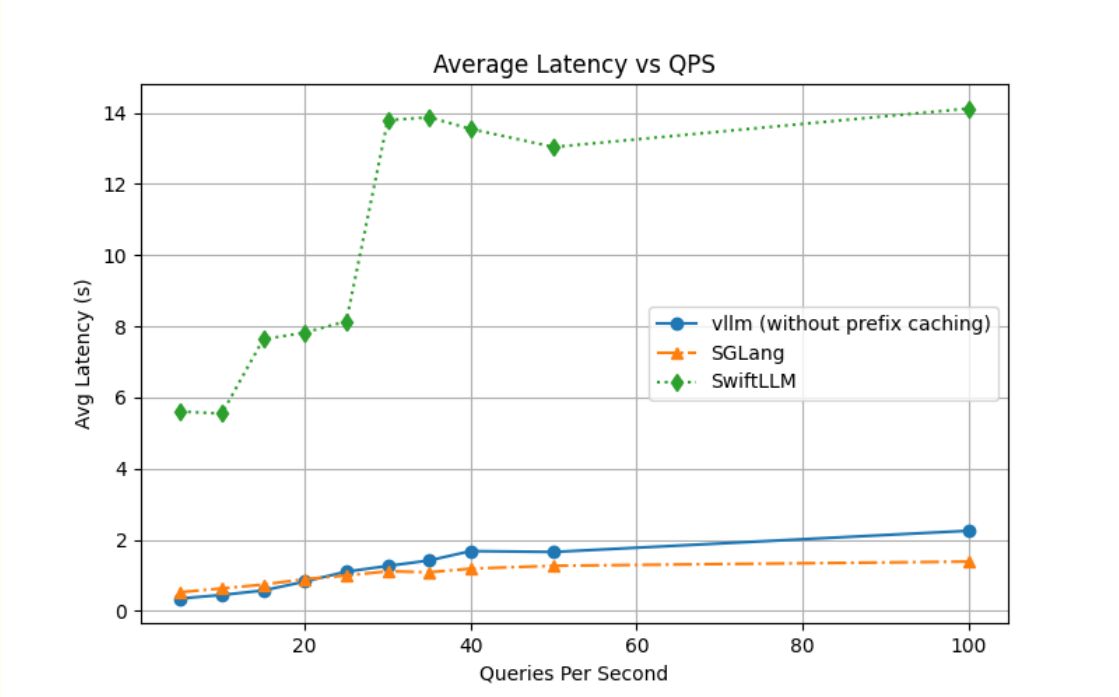


Figure: Average Latency comparison across varying workloads

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

LLM Request Timelines

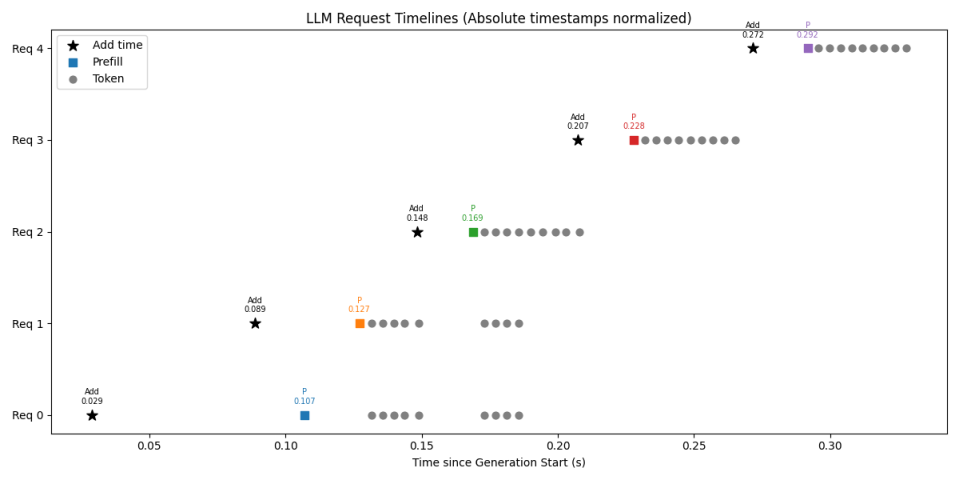


Figure: 20 Requests per second — without chunked prefill — vLLM V0 Engine

- Prefills take too much time as compared to decodes, primiarily because of large system prompt size.
- In this mode prefills can't go along with decodes.
- Prefills are preferred over decodes to reduce TTFT but total response time is quite high.

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

NSYS Measurements



Figure: NSYS Metrics Snapshot

- For compute: we just check SM Active and Warp occupancy.
- For memory: vLLM books all the available memory in the start, scheduler stats show the actual KV Cache usage.
- PCIe Bandwidth isn't utilized fully - small workloads - no need to transfer stuff to CPU.

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

Throughput vs Load for vLLM

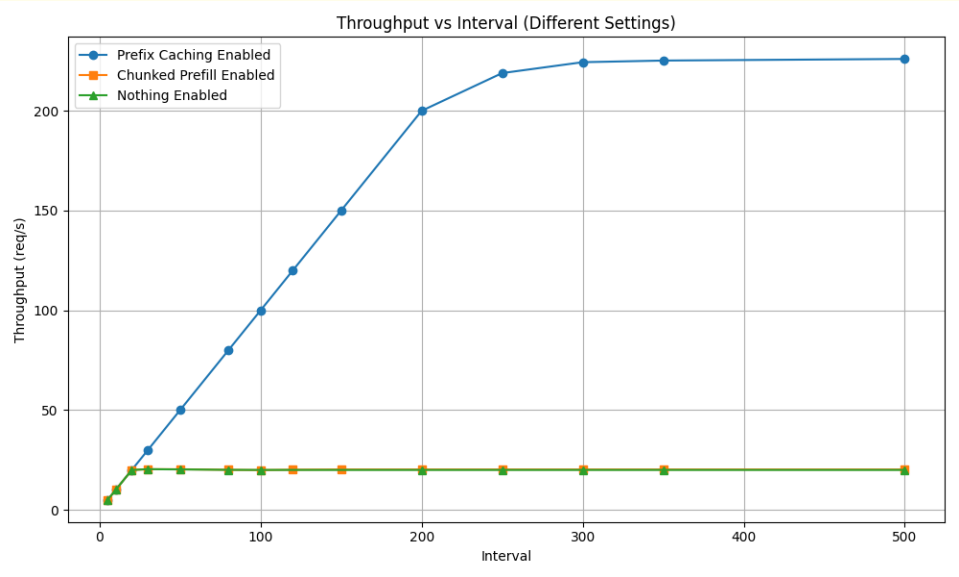


Figure: Throughput vs Load

- Chunked prefill does not really help that much for our type of workload.
- Prefix caching helps a lot, as is visible from the immense difference in throughputs.
- By changing the batch size (in terms of number of sequences and the token budget), we can achieve higher throughputs too.

Systems for LLMs
Authors
Workload Picked
Batch of requests through various Engines
LLM Request Timelines
NSYS Measurements
Throughput vs Load for vLLM
GPU Memory Breakdown
Future Steps

GPU Memory Breakdown



Component	Memory Occupied
Weights	0.93 GiB
Peak Activation	4.07 GiB
Non-Torch	0.02 GiB
CUDA Graph	0.41 GiB
KV Cache: Intersection (Prefix)	1,280 tokens / 0.0157 GiB
KV Cache: Extras for Requests	640 tokens (10 requests × 64 tokens) / 0.0078 GiB
KV Cache: Free	2,672,880 tokens / 32.8445 GiB

Table: GPU Memory Breakdown and KV Cache Usage

Future Steps



Systems for LLMs

Authors

Workload Picked

Batch of requests through various Engines

LLM Request Timelines

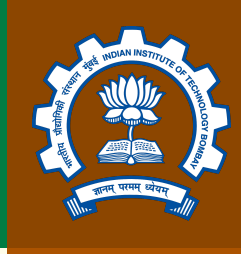
NSYS Measurements

Throughput vs Load for vLLM

GPU Memory Breakdown

Future Steps

- Find the max throughput we can achieve for this kind of workload through vLLM
- Find the throughput for a family of requests, each family sharing a system prompt amongst its members
- Explore other batching techniques for such a family of requests, taking prefix sharing into consideration
- Possible changes to the scheduling algorithm
- Can we offload stuff to CPU, when we have computed the KV Cache, and we just need to perform decode operations?
- Possible optimizations to fine tuning



Systems for LLMs

Authors

Workload Picked

Batch of requests through various Engines

LLM Request Timelines

NSYS Measurements

Throughput vs Load for vLLM

GPU Memory Breakdown

Future Steps

Thank You